# SPARFUCHS

Corporation®

Warp-speed delivery. Fort Knox security. 30 days.

USER GUIDE | VOLUME I

# Sparfuchs QA End-User Guide

## For operators running canaries, orchestrated reviews, training, and docs flows

| | |
|---|---|
| Version | 1.0 |
| Release date | April 22, 2026 |
| Audience | For operators running canaries, orchestrated reviews, training, and docs flows |
| Authors | Sparfuchs Corporation Documentation Team |
| Classification | Corporate Confidential |

No part of this document, or any associated materials, may be reproduced, distributed, transmitted, displayed, published, or broadcast in any form or by any means, including electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of Sparfuchs Corporation.

Unauthorized use of this material may violate copyright laws, trademark laws, and other applicable regulations. All trademarks and registered trademarks mentioned herein are the property of their respective owners.

# How to use this guide

This guide is organized around real tasks, not features. Every chapter answers a question that an operator would actually ask, from "How do I run my first QA review?" to "How do I investigate a flagged finding that I do not understand?"

Each procedure follows the same shape. Prerequisites come first. Numbered steps come next. Expected outcomes and a "what happens next" note close the procedure. Warnings and cautions are called out in coloured boxes. Code is set in a monospaced face with a light grey background so it is easy to copy from the web version.

## Conventions

**Operator.** The person running Sparfuchs QA from a local workstation. For the purposes of this guide, operator and user are interchangeable.

**Target repo.** The external repository being reviewed. Always a local git working copy unless *ACCEPT_NO_GIT=1* is passed.

**Review.** A single invocation of *make qa-review* or one of its siblings. Reviews produce findings, a report, and optional generated tests.

**Finding.** A single reported issue attached to a file and a severity.

## Accessibility and navigation

The PDF ships with a full table of contents, a list of figures, and PDF bookmarks that mirror the chapter structure. Screen readers can follow the heading outline. Every command example is plain ASCII, no decorative glyphs, so copy-paste works from both the PDF and the web version.

## Feedback

If something is unclear, wrong, or missing, please file an issue in the sparfuchs-qa repository with the label *docs*, or email the documentation owner listed on the cover page. Every page includes a "Was this helpful?" control in the web version.

# Table of contents


How to use this guide 2

List of figures 5

1. Introduction and overview 6

1.1 What Sparfuchs QA is for . . . 6

1.2 Capabilities at a glance . . . 6

1.3 What you need before you start . . . 8

2. Getting started 9

2.1 Quick start, five minute path . . . 9

2.2 Your first review, step by step . . . 9

2.3 Interface tour . . . 11

3. Core workflows 12

3.1 Running canaries . . . 12

3.2 Running a QA review . . . 13

3.3 Running a full audit . . . 15

3.4 Reviewing an authenticated application . . . 15

3.5 Running a targeted subset of agents . . . 16

3.6 Hunting stubs, mocks, and dead code . . . 16

3.7 Generating training materials and docs . . . 16

3.8 Working with the file audit cache . . . 16

3.9 Deltas, evolution, and cleanup . . . 17

4. Advanced usage and customization 18

4.1 Switching providers and engines . . . 18

4.2 Tuning coverage strategy . . . 18

4.3 Per-agent overrides . . . 19

4.4 Data classification . . . 19

4.5 Module scoping for large monorepos . . . 19

4.6 Writing your own canary . . . 20

4.7 Watch mode . . . 20

5. Monitoring and oversight of agents 21

5.1 Tracking a run in progress . . . 21

5.2 Reading session logs . . . 21

5.3 Finding lifecycle . . . 21

5.4 Intervening and pausing . . . 22
5.5 Performance and token metrics . . . 22
5.6 Alerting when agents misbehave . . . 22
6. Troubleshooting 24
6.1 Installation and launch errors . . . 24
6.2 Canary failures . . . 25
6.3 Agent skipped or stuck . . . 25
6.4 Permission prompts that won't end . . . 25
6.5 Cleanup issues . . . 25
6.6 When a finding looks wrong . . . 26
6.7 Error message glossary . . . 26
7. Reviewing and acting on reports 27
7.1 Report bundle at a glance . . . 27
7.2 A reading order that works . . . 27
7.3 The release gate verdict . . . 27
7.4 Generated tests and the graduation ratchet . . . 28
8. Frequently asked questions 30
9. Glossary 32
10. Appendices 34
Appendix A. Command reference . . . 34
Appendix B. Useful environment variables . . . 34
Appendix C. Printable pre-push checklist . . . 35
Appendix D. Support and feedback . . . 35
Appendix E. Changelog . . . 35
Appendix F. Documentation maintenance plan . . . 36

# List of figures


| # | Figure | Chapter |
|---|---|---|
| 1 | Figure 1.1 Sparfuchs QA, system overview | Chapter 1 |
| 2 | Figure 2.1 Choosing a review mode | Chapter 2 |
| 3 | Figure 3.1 Canary taxonomy | Chapter 3 |
| 4 | Figure 3.2 Agent execution loop | Chapter 3 |
| 5 | Figure 5.1 Finding lifecycle | Chapter 5 |
| 6 | Figure 6.1 Troubleshooting decision tree | Chapter 6 |
| 7 | Figure 7.1 Test generation lifecycle | Chapter 7 |

# 1. Introduction and overview

Sparfuchs QA is a TypeScript multi-agent quality toolkit. It wraps a family of specialist AI agents, a suite of static canary checks, a credential-aware test harness, and a local reporting layer into a single operator-friendly CLI. You point it at any git repository, pick a review mode, and receive a markdown report with ranked findings, generated tests, and a Go or No-Go release verdict.

**At a glance. What makes Sparfuchs QA different.** Most code-scanning tools hand you a list of findings and leave the "should we ship?" decision to you. Sparfuchs QA delivers a verdict: **PASS**, **NEEDS CHANGES**, or **BLOCKED**, backed by a transparent risk score and a human override. Details in chapter 7.

## 1.1 What Sparfuchs QA is for

Teams use Sparfuchs QA in three places in the software delivery lifecycle. It runs as a pre-push guard on the operator's machine, where the fast canaries catch regressions before they ever reach CI. It runs as a pre-PR review against a working branch, where the orchestrated agents look at code quality, security, RBAC, and accessibility. And it runs as a full audit at project inception or at periodic health checks, where every agent runs across the whole codebase.

The toolkit is not a replacement for human code review, for penetration testing, or for integration tests. It is a force multiplier for human reviewers: it surfaces hundreds of findings quickly, groups them by severity and file, and leaves the judgment about how to act on them to a person.

### 1.1.1 When Sparfuchs QA is not the right tool

Sparfuchs QA is a focused tool, not a silver bullet. Please do not adopt it for any of the situations below, these are better served by purpose-built products:

- **Runtime vulnerability testing (DAST).** Sparfuchs QA is a static analyzer. It reads source; it does not execute your app or probe a running endpoint.
- **Penetration testing or adversarial red-team work.** The agents are not adversarial. They will not try to bypass your controls.
- **Business logic correctness.** Agents can find bugs in code patterns, but they cannot verify that your pricing logic applies the correct discount or that a state machine transitions in the correct order. That still requires hand-authored test cases with known expected values.
- **Repos that contain hardcoded PII.** Secret redaction is in place; PII redaction is not. Wait for the v1.1 roadmap item, or run with *dataClassification: restricted* to keep source off external APIs.
- **Compliance certification (SOC 2, HIPAA, PCI-DSS).** The toolkit is a tool, not a certifying body. It can produce evidence for an auditor, not the certificate itself.

## 1.2 Capabilities at a glance

The platform combines deterministic static checks (canaries) with agent-driven deep analysis (reviewers, fuzzers, verifiers). The diagram below shows the main components and the flow of artifacts. Keep it in mind as you read the rest of the guide, every later chapter touches one or more of these components.

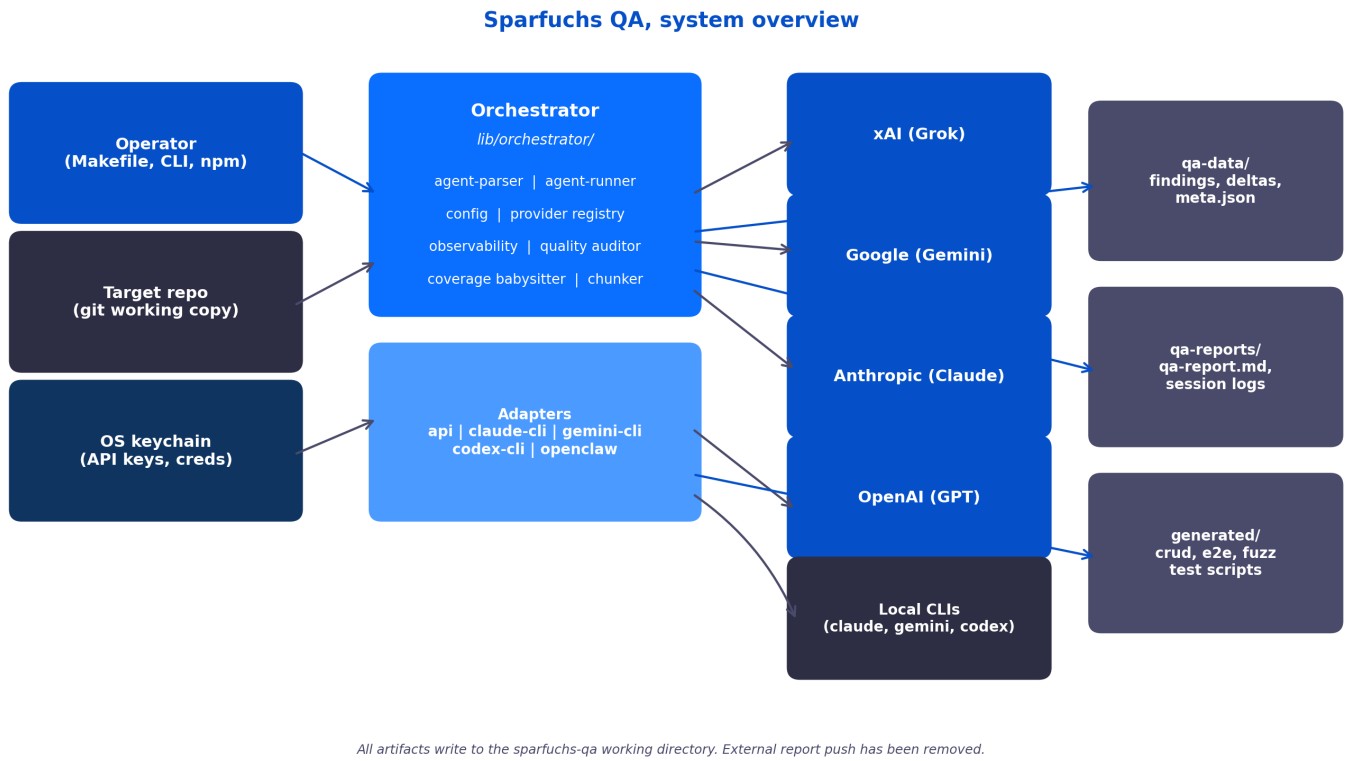


*Figure 1.1 Sparfuchs QA, system overview.*

### 1.2.1 What is an agent, concretely?

A Sparfuchs QA **agent** is a specialist defined as a markdown file in *.claude/agents/*, with a prompt, a model tier, and an allow-list of tools (*Read*, *Grep*, *Bash*, and so on). When a review runs, the orchestrator deploys the relevant agents into a temporary *.claude/agents/* directory in the target repo, invokes an LLM per agent, collects each agent's JSON output, and removes the agent files at cleanup. You can read, edit, or add agents; they are plain text files, not compiled code. The repository currently ships over 40 specialist agents (see the Admin Guide for the exact breakdown), plus 2 optional documentation agents.

### Autonomy levels

Sparfuchs QA is a human-in-the-loop system. The orchestrator can act autonomously inside an agent's narrow domain (read files, run grep, call an LLM) but it never writes to the target repo, never commits code, and never pushes changes. Generated tests are written into a separate *generated/* directory in the sparfuchs-qa workspace, and they are only executed when a human copies them into the target project's test suite.

### User personas

Three personas shape this guide:

- **Novice end-user.** Runs *make qa-quick* before pushing. Reads only chapters 1 through 3 and dips into chapter 6 when something fails.
- **Power operator.** Runs the full multi-agent review against arbitrary repos, cares about which provider is serving which agent, and tunes *config/models.yaml*.
- **IT administrator.** Manages credentials, keychain secrets, provider budgets, and access. Most of the IT administrator's material lives in the companion Admin Guide, but chapter 5 of this guide covers the monitoring and oversight that end-users need.

### Assumptions

You are comfortable using a Unix-style terminal, *git*, and *make*. You can install Node.js packages with *npm*. You do not need prior experience with LLM orchestration or with the specific provider APIs.

## 1.3 What you need before you start

Before your first review, confirm the core prerequisites and choose one path for reaching an LLM. The core prerequisites are non-negotiable; the LLM path is a choice.

### 1.3.1 Core prerequisites (required)

| Requirement | Check | Notes |
|---|---|---|
| Node.js 22 or newer | `node --version` | Enforced in *package.json* engines. |
| npm (workspace-aware) | `npm --version` | Ships with Node. |
| Git | `git --version` | Required for target repo analysis. |
| Make | `make --version` | Recommended; you can substitute direct npm scripts. |
| Target repo, git-backed | `cd /path && git status` | Or pass ACCEPT_NO_GIT=1. |
| Disk | ~300 MB | node_modules + qa-data artifacts. |

### 1.3.2 Choose exactly one LLM path

You need one, and only one, of these three options to reach an LLM. Pick the one that fits your environment.

1. **Claude Code CLI plus an Anthropic credential** (simplest). Install with *npm install -g @anthropic-ai/claude-code* and authenticate on first run. Works with *make qa-review* out of the box, no extra flags.
2. **An API key for xAI, Google, OpenAI, or Anthropic**, stored in the OS keychain under service *sparfuchs-qa*. Use with *ENGINE=orchestrated PROVIDER=<name>*.
3. **A locally installed alternate CLI**: *gemini*, *codex*, or *openclaw*. Authenticate the CLI once, then use *ENGINE=orchestrated PROVIDER=gemini-cli* (or similar).

**Note.** The toolkit never installs itself inside your target repo. All state lives in the sparfuchs-qa working directory under *qa-data/*, *qa-reports/*, and *generated/*. Uninstalling is as simple as deleting the sparfuchs-qa checkout.

# 2. Getting started

This chapter walks you from a fresh clone to your first successful review in about five minutes. If you already have Node 22 and a working LLM credential, the whole thing is four commands.

## 2.1 Quick start, five minute path

Run the following in order, from a fresh terminal:

```
# 1. Clone
git clone https://github.com/Sparfuchs-Pro/sparfuchs-qa.git
cd sparfuchs-qa

# 2. Install dependencies
make qa-setup

# 3. Run the canary suite against this repo (sanity check)
make qa-quick

# 4. Run your first review against a target repo
make qa-review REPO=/absolute/path/to/your/project
```

If step 3 prints **15/15 canaries passed**, the toolkit itself is healthy. If step 4 completes and writes a report under *qa-reports/*, you are done. The rest of this guide helps you work faster and go deeper.

## 2.2 Your first review, step by step

### Prerequisites

- Node 22 or newer is on your PATH.
- You can run *make* and *npx*.
- At least one LLM credential exists, either in the OS keychain or as an environment variable.
- The target repo is a git repository.

### Step 1, pick a review mode

The review mode determines how much work the orchestrator does. The three most common choices are **build check** (fastest), **diff review** (default), and **full audit** (most thorough). The decision tree below shows how to choose.

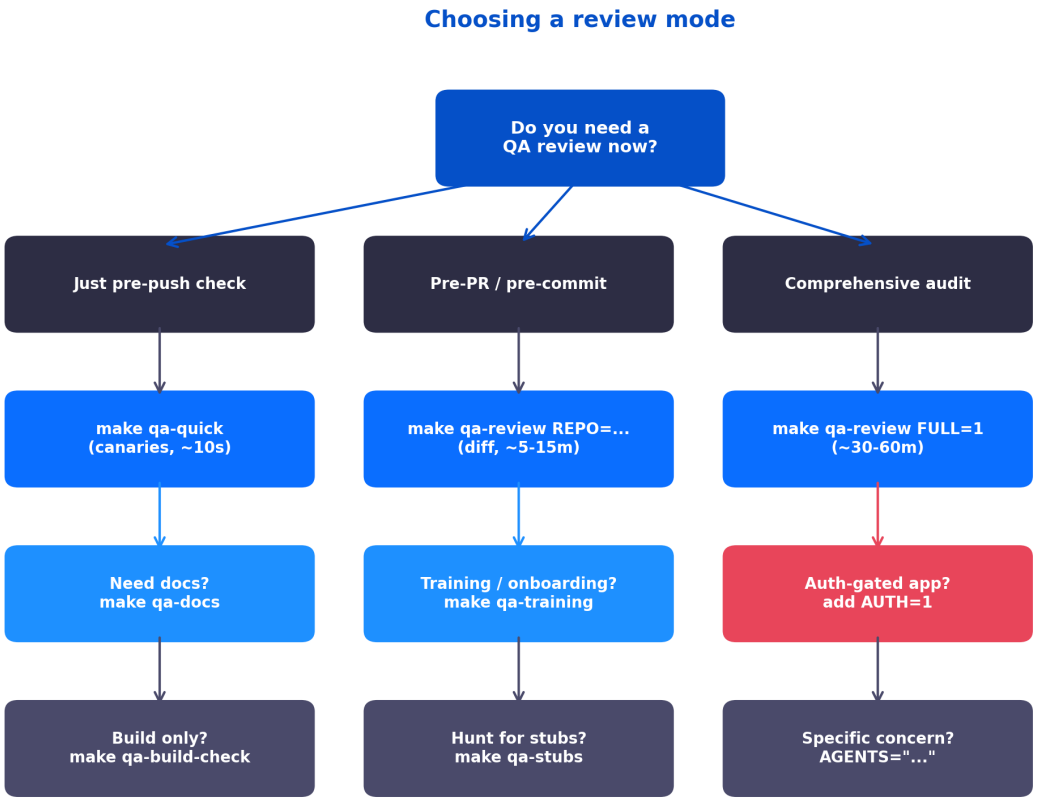


*Figure 2.1 Choosing a review mode.*

## Step 2, store a credential

The orchestrator needs at least one provider to run agents. The recommended path is the OS keychain, which encrypts keys at rest and keeps them out of shell history. On macOS:

```
security add-generic-password \
-s sparfuchs-qa -a ANTHROPIC_API_KEY -w 'sk-ant-...'

# Confirm
make qa-keys-check
```

If you prefer a one-shot environment variable, export the key and move on. Env vars work but are not persisted across shells.

## Step 3, launch the review

```
make qa-review \
REPO=/path/to/project \
FULL=1 \
PROJECT="My App" \
PERSON="Alice"
```

The orchestrator prints a short banner, deploys the relevant specialist agents into the target repo's *.claude/agents/* directory, and starts the run. Sparfuchs QA ships 40+ specialist agents (see section 1.2.1); which ones deploy depends on your review mode and the testability preflight. A live status table shows which agent is running, which provider it is using, and whether any agents were skipped.

### Step 4, approve permission prompts

Because Claude is launched with *--permission-mode default*, it will ask before reading files for the first time, running bash commands, and writing to the *qa-reports/* directory. You can approve individually or press **a** to allow all reads for the session. Approve the read permissions. Deny anything you are not comfortable with, the agent will skip and move on.

> **Tip.** On long runs, leave the terminal visible. You will see a friendly activity log for each agent. If a run appears idle for more than two or three minutes, check the latest session log file (see chapter 5) to see what the active agent is doing.

### Step 5, read the report

When the run ends, the orchestrator restores the target repo to its original state (see chapter 7) and writes the report to the sparfuchs-qa workspace:

```
sparfuchs-qa/qa-reports/
2026-04-22_my-app_qa-report.md # findings by severity
2026-04-22_my-app_session-log.md # full agent transcripts
2026-04-22_my-app_spec-report.md # spec and PRD verification
2026-04-22_my-app_qa-gaps.md # coverage gap analysis
```

Open the *qa-report.md* first. It is grouped by severity (Critical, High, Medium, Low) and then by file. Each finding has a description, a suggested fix, and the agent that produced it.

### Expected outcome

A successful first review ends with a **verdict** of *PASS*, *NEEDS CHANGES*, or *BLOCKED* printed at the end of the report, along with a summary count of findings. The target repo is left exactly as it was before the run, including its *.claude/agents/* directory.

## 2.3 Interface tour

Sparfuchs QA is a CLI-first tool. The operator surface has four entry points that all funnel into the orchestrator:

| Entry point | When to use it | Example |
|---|---|---|
| **Makefile** | Everyday operation | `make qa-review REPO=...` |
| **npm scripts** | CI / automation | `npm run qa:quick` |
| **Direct tsx** | Debugging a canary or script | `npx tsx canaries/index.ts` |
| **Shell wrapper** | The orchestrator itself | `bash scripts/qa-review-remote.sh` |

> **Note.** The live status table shown during orchestrated runs is a small TTY renderer in *lib/orchestrator/renderers/tty-renderer.ts*. If you prefer a log-only view, set *RENDERER=log* in your environment.

# 3. Core workflows

This is the longest chapter in the guide, because it is where the toolkit earns its keep. Each section is a complete procedure: prerequisites, numbered steps, expected outcome, and a "what happens next" note. Skim the titles to find the workflow you need.

## 3.1 Running canaries

Canaries are the fastest layer of Sparfuchs QA: deterministic, static checks that finish in roughly ten seconds. They are the minimum gate before a push.

### 3.1.1 Prerequisites

- You have run *make qa-setup* at least once.
- You are in the sparfuchs-qa repo root, or *TARGET_REPO* is set.

### 3.1.2 Procedure

1. Open a terminal in the sparfuchs-qa checkout.
2. Run *make qa-quick*.
3. Watch the JSON summary stream to stdout.
4. Confirm every canary's *passed* field is *true*. If any fail, read the *hint* field for a one-line explanation.

### 3.1.3 Expected outcome

```
{
"runId": "run-1714231200000",
"timestamp": "2026-04-22T10:00:00.000Z",
"results": [
{ "id": "hardcoded-credential", "passed": true, "value": 0, ... },
{ "id": "rbac-bypass", "passed": true, "value": 0, ... },
{ "id": "stale-closure", "passed": true, "value": 0, ... },
...
]
}
```

### 3.1.4 What happens next

If every canary passes, you are cleared to push. If any fails with severity **critical** or **high**, fix the underlying issue before pushing. See chapter 6 for decision trees on common canary failures.

### 3.1.5 Canary taxonomy

*Figure 3.1 Canary taxonomy. Each group enforces a distinct invariant.*

## 3.2 Running a QA review

A QA review is an orchestrated run of the 40+ specialist agents that ship with Sparfuchs QA. The orchestrator chooses which agents apply based on the file mix of the target repo, the review scope, and the testability preflight.

### 3.2.1 Prerequisites

- A git-backed target repo. If the repo is not under git, pass *ACCEPT_NO_GIT=1*.
- At least one LLM credential is available (keychain or env var).
- The Claude Code CLI is installed and authenticated, **or** you pass *ENGINE=orchestrated PROVIDER=codex-cli* (or another supported alternative).

### 3.2.2 Procedure

**1.** Decide between diff and full audit. Diff is the default and reviews only files that have changed. Full audit scans every source file.

**2.** Decide whether you need authenticated testing. Set *AUTH=1* to trigger the credential setup wizard.

**3.** Optionally scope to a subdirectory with *MODULE="apps/web"* for a large monorepo.

**4.** Launch: *make qa-review REPO=/path/to/project [FULL=1] [AUTH=1]*.

**5.** Approve permission prompts for file reads and bash commands.

**6.** Wait for the orchestrator to finish. The expected duration is 5 to 60 minutes depending on scope and chosen providers.

**7.** Read the report in *sparfuchs-qa/qa-reports/*.

**Warning.** Do not run a review against a repo you do not own or have permission to analyze. Source files are sent to the configured LLM provider. Set *dataClassification: restricted* in *config/models.yaml* to disable external API providers entirely. See the Admin Guide for governance controls.

### 3.2.3 Agent execution loop

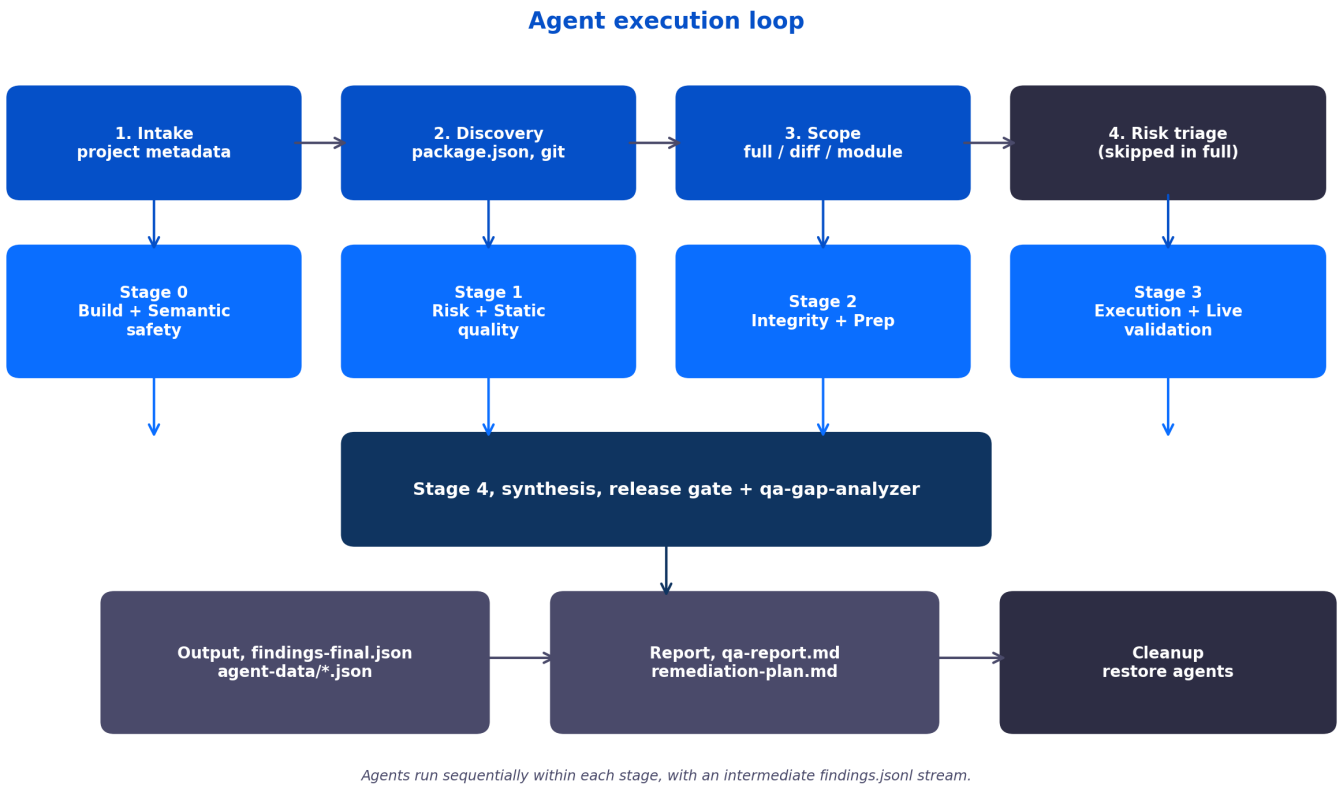


*Figure 3.2 The orchestrator's execution loop from intake to cleanup.*

The five stages in the diagram above have distinct purposes. Understanding them helps you read the live status table and the final report.

| Stage | Purpose | Representative agents |
|---|---|---|
| Stage 0 | Build and semantic safety. Does the code compile and behave as a human would expect after recent changes? | build-verifier, semantic-diff-reviewer |
| Stage 1 | Risk and static quality. The bulk of the review: security, RBAC, code quality, compliance, accessibility. | security-reviewer, code-reviewer, rbac-reviewer, a11y-reviewer (+13 more) |
| Stage 2 | Integrity and test prep. Schema drift, mock integrity, and generation of candidate CRUD / E2E / fuzz tests. | schema-migration-reviewer, mock-integrity-checker, crud-tester, boundary-fuzzer |
| Stage 3 | Execution and live validation. Actually runs tests; probes APIs; analyzes failures. | test-runner, smoke-test-runner, api-contract-prober, failure-analyzer |
| Stage 4 | Synthesis and the gate. Aggregates every earlier output into a verdict. | qa-gap-analyzer, release-gate-synthesizer |

### 3.2.4 Expected outcome

The run writes a bundle of artifacts to *qa-data/<project>/runs/<runId>/* and a human-facing set of markdown reports to *qa-reports/*. Every run guarantees the following, even on partial failure:

- *meta.json* with a *status* of *succeeded*, *partial*, or *errored*.
- *findings-final.json* with the deduplicated, canonical finding list.
- *delta.json* comparing this run to the previous one.
- *coverage-report.json* and *quality-audit.json*.
- Per-agent handoff envelopes under *agent-data/*.
- Human reports: *qa-report.md*, *remediation-plan.md*, *observability-gaps.md*, *qa-gaps.md*, *delta-report.md*.

### 3.2.5 What happens next

Your first read-through should focus on **Critical** findings in *qa-report.md*. Sort by file, group by severity, and pick the three or four biggest-impact items. Use the *remediation-plan.md* as a fix checklist, not as a definitive to-do list.

## 3.3 Running a full audit

A full audit scans every source file with every applicable agent. It is the right mode for first-time onboarding a project or for monthly health checks. Expect the run to take 30 to 60 minutes on a medium codebase.

Full audit differs from diff review in four ways:

- Every source file is considered, not just the git diff.
- Every agent runs. There is no risk triage skipping step.
- The testability scanner may split the work into chunks of roughly 25 files each.
- The file audit cache (see section 3.8) is updated for every file.

```
make qa-review REPO=/path/to/project FULL=1
```

## 3.4 Reviewing an authenticated application

When the target application has authenticated endpoints (login, signed APIs, role-gated pages), pass *AUTH=1* so that CRUD, E2E, and contract agents can generate tests that actually log in.

### 3.4.1 The setup wizard

The wizard is interactive and supports five strategies. Pick the one that matches your target app.

| Strategy | Use when | Wizard asks for |
|---|---|---|
| Email + Password | Firebase, standard login forms | Email, password, Firebase API key |
| API Token | Bearer token already on hand | Token |
| OAuth Token | Pre-obtained OAuth access token | Access token, refresh token, expiry |
| Basic Auth | HTTP Basic | Username, password |

| Strategy | Use when | Wizard asks for |
|---|---|---|
| None | Public API only | Nothing, creates a no-op file |

**Warning.** The credential file is written as plaintext JSON to */tmp/sparfuchs-qa-creds-{runId}.json* with mode 0600. It is deleted automatically on cleanup. Do not use production credentials here. Create a dedicated test account instead.

## 3.5 Running a targeted subset of agents

Sometimes you want a focused pass, for example "security only" or "RBAC and permissions only." Use *AGENTS* with a comma-separated list:

```
make qa-review \
REPO=/path/to/project \
AGENTS="security-reviewer,rbac-reviewer,access-query-validator"
```

The orchestrator still runs the intake, discovery, and scope stages, but only the named agents execute in stage 1. This is the fastest way to interrogate a specific dimension without waiting for the full sweep.

## 3.6 Hunting stubs, mocks, and dead code

Stubs pretending to be real features are a common source of production bugs. The *stub-detector*, *dead-code-reviewer*, and *mock-integrity-checker* agents triangulate this risk.

```
make qa-stubs REPO=/path/to/project
```

Use this when an engineer hands off a feature that "works locally" but you suspect parts of the stack are mocked. The report will list each suspected stub with a confidence score and a pointer to the real implementation if one exists.

## 3.7 Generating training materials and docs

Sparfuchs QA ships with two documentation-oriented agents: *training-system-builder* and *architecture-doc-builder*. They are deployed only when *--training* or *--docs* is set.

```
# Training materials
make qa-training REPO=/path/to/project
make qa-training REPO=/path/to/project MODULE="auth"
make qa-training REPO=/path/to/project JOURNEY="onboarding"

# Architecture docs
make qa-docs REPO=/path/to/project
make qa-docs-all REPO=/path/to/project
```

**Tip.** Training mode is the fastest way to onboard a new engineer onto an unfamiliar codebase. It produces module-level decision trees, glossaries, and quiz questions derived from the actual source code.

## 3.8 Working with the file audit cache

Full audits are expensive. The file audit cache tracks which files were audited at which commit, so subsequent runs can skip unchanged files. The cache is stored per project under *qa-data/<project>/file-audit.json*.

| Command | Effect |
|---|---|
| `make qa-cache-status PROJECT=my-app` | Show which files are cached and at which commit. |
| `make qa-cache-reset PROJECT=my-app` | Clear the cache so the next run audits everything. |

**Note.** The cache key is (file path, content hash, commit sha). If you switch branches frequently or rebase, expect cache misses. A cache reset before a monthly audit is a good hygiene habit.

## 3.9 Deltas, evolution, and cleanup

After two or more runs for the same project, you can compare them. These three commands round out the day-to-day cycle:

| Command | What it does |
|---|---|
| `make qa-delta PROJECT=my-app` | Compares the latest run to the previous one: new, recurring, remediated. |
| `make qa-evolve-v2 PROJECT=my-app` | Analyzes patterns across historical runs and suggests threshold adjustments. |
| `make qa-cleanup PROJECT=my-app` | Archives old runs beyond the most recent ten. |

# 4. Advanced usage and customization

## 4.1 Switching providers and engines

By default, Sparfuchs QA runs in *direct* mode with the Claude CLI. That is the simplest path and the one we recommend for first-time use. For more control, switch to the *orchestrated* engine and pick a provider explicitly.

```
# Use Google Gemini through the orchestrated engine
make qa-review REPO=/path/to/project \
ENGINE=orchestrated PROVIDER=google

# Use the Codex CLI locally
make qa-review REPO=/path/to/project \
ENGINE=orchestrated PROVIDER=codex-cli
```

The orchestrated engine uses *config/models.yaml* to resolve the model tier (*heavy*, *mid*, or *light*) for each agent. The fallback chain is consulted when the primary provider is unavailable. In orchestrated mode, the live status table reports the resolved provider for every agent row, including skipped agents.

### 4.1.1 Understanding run cost

This is the first question every engineering manager asks. Actual cost depends on provider pricing, model tier, and the file mix of your repo, but the following ranges are realistic for a full audit with default settings:

| Codebase size | balanced | thorough | exhaustive |
|---|---|---|---|
| Small (~10k LOC) | $5 to $15 | $15 to $30 | $25 to $60 |
| Medium (~50k LOC) | $15 to $40 | $40 to $80 | $40 to $100 |
| Large (~250k LOC) | $100 to $250 | $200 to $450 | $300 to $700 |
| Very large (~1M+ LOC) | Use MODULE scoping, see section 4.5 | | |

Diff reviews cost a fraction of a full audit because only changed files are analyzed. Canary runs are free (no LLM). The *lite* preset (see chapter 4) typically drops cost by 60 to 80 percent versus *full*. Monitor actual spend via the per-agent *tokensIn* and *tokensOut* fields in *quality-audit.json*, and cap runs by setting *tokenBudget.defaultCap* in *config/models.yaml*.

> **Tip.** Pinning honesty-critical agents to Grok (see section 4.3) is usually cheaper than the Anthropic heavy tier while scoring similarly on honesty benchmarks. The *standard* and *lite* presets are the easiest way to control spend without giving up the release gate verdict.

## 4.2 Tuning coverage strategy

*config/models.yaml* exposes a **coverageStrategy** knob that trades analysis depth for cost and latency:

| Strategy | Approx. coverage | When to use |
|---|---|---|
| sweep | ~40% | Pre-commit, pre-push, fast feedback |
| balanced | ~65% | Default, pre-PR reviews |
| thorough | ~85% | Monthly audits |
| exhaustive | ~95% | Quarterly health checks, first-time onboarding |

## 4.3 Per-agent overrides

You can lock a specific agent to a specific provider or tighten its step budget. Edit the *agentOverrides* block in *config/models.yaml*:

```
agentOverrides:
security-reviewer:
provider: xai
qa-gap-analyzer:
provider: xai
ui-intent-verifier:
provider: xai
disableBash: true # Read/Grep/Glob only, highest lockdown
observability-auditor:
provider: xai
maxSteps: 75 # 6-dimension audit needs more steps
ref-doc-verifier:
provider: xai
maxSteps: 100 # verifying many claims against code
```

**Note.** Grok is the recommended provider for honesty-critical agents (security, gap analysis, release gate). This is configurable but not arbitrary: agent quality is measured against a golden corpus before defaults are changed.

## 4.4 Data classification

The **dataClassification** setting controls which providers can receive repo content. This is the easiest way to lock down a sensitive project:

| Level | API providers | CLI providers | Notes |
|---|---|---|---|
| public | All enabled | All enabled | Default. |
| internal | Only approvedProviders list | All enabled | Set *approvedProviders*. |
| restricted | Disabled | All enabled | No data leaves the host via API. |

## 4.5 Module scoping for large monorepos

If your target repo is a monorepo with dozens of apps, reviewing the whole thing every time is slow. Use *MODULE* to scope the run to a subdirectory:

```
make qa-review REPO=/path/to/monorepo \
ENGINE=orchestrated PROVIDER=codex-cli \
MODULE="apps/web"
```

Module scoping is especially useful when combined with *AGENTS*. A targeted pass like "run *security-reviewer* and *rbac-reviewer* on *apps/api/*" finishes in minutes.

## 4.6 Writing your own canary

Sparfuchs QA canaries are auto-discovered: any **.canary.ts* file in *canaries/* is loaded at startup. Drop a new file in and it runs next time.

1. Create *canaries/my-check.canary.ts*.
2. Export a default async function that returns *{ pass, severity, hint, value, threshold, trend }*.
3. Keep the execution under 500 milliseconds.
4. Do not hit external services: read files, run git, shell out to local tools only.
5. Run *make qa-quick* to confirm your canary is picked up.

```
// canaries/my-check.canary.ts
import { execSync } from 'child_process';

export default async function myCheck() {
const count = /* your measurement */ 0;
const threshold = 5;
return {
id: 'my-check',
projectId: process.env.PROJECT || 'sample-project',
type: 'code-quality',
severity: count > threshold ? 'high' : 'info',
hint: `Found ${count} items; threshold is ${threshold}`,
value: count,
threshold,
passed: count <= threshold,
trend: 'stable',
lastSeen: new Date().toISOString(),
history: [],
};
}
```

## 4.7 Watch mode

During iterative work, the watch flag re-runs canaries whenever files change. Useful when hardening a single file.

```
make qa-watch # watch the current repo
TARGET_REPO=/path npx tsx canaries/index.ts --watch
```

# 5. Monitoring and oversight of agents

Sparfuchs QA is an agentic system. Agents make thousands of independent decisions during a run: which files to read, which tools to call, which findings to raise, which to suppress. This chapter shows how to watch, intervene, and audit those decisions in real time and after the fact.

## 5.1 Tracking a run in progress

During an orchestrated run, the TTY renderer in *lib/orchestrator/renderers/tty-renderer.ts* prints a live status table with one row per agent. Columns include name, provider, status, progress, tokens used, and findings raised so far.

If you prefer machine-readable output (for CI, log forwarding, or an external dashboard), set *RENDERER=json* or *RENDERER=log*. The JSON renderer emits one event per line. The log renderer is a terse one-line-per-transition format.

```
RENDERER=json make qa-review REPO=/path/to/project \
ENGINE=orchestrated PROVIDER=google 2>status.jsonl
```

## 5.2 Reading session logs

After the run ends, the full agent transcript is written to *qa-reports/<date>_<project>_session-log.md*. This is the authoritative record of what every agent saw, what it said, and which tools it called. Use it to:

- Confirm a finding that seems suspicious. If the agent reported a bug on a file, search the session log for that file name and read the surrounding paragraph.
- Diagnose a skipped agent. Each skip is logged with a reason.
- Audit tool use. Every bash command the agent ran is in the log.

## 5.3 Finding lifecycle

Findings move through seven possible states, persisted in *qa-findings-manager*. Understanding the lifecycle is key to interpreting a delta report.

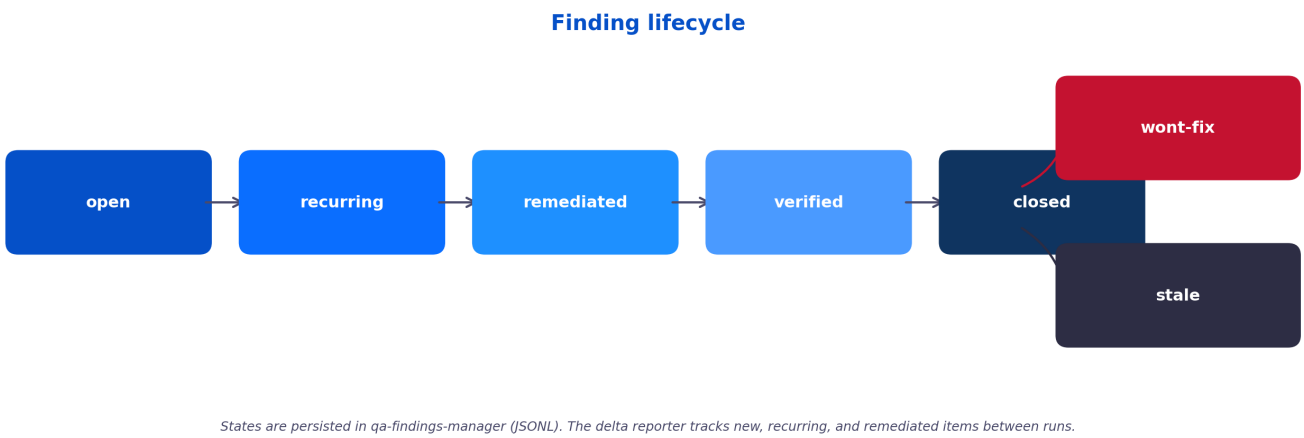


*Figure 5.1 Finding lifecycle.*

| State | Meaning |
|---|---|
| open | Newly detected in the most recent run. |
| recurring | Appears in two or more consecutive runs. |
| remediated | Developer has implemented a fix. |
| verified | The fix has been validated by a subsequent run. |
| closed | Finding is fully resolved. |
| wont-fix | Intentionally not addressed. Must include justification. |
| stale | No longer relevant, for example the file has been deleted. |

## 5.4 Intervening and pausing

If a run is going wrong, press *Ctrl-C*. The orchestrator installs a cleanup trap: it will attempt to remove deployed agent files from the target repo, restore any backed-up agents, and delete the temp credential file. Partial reports remain in *qa-reports/* so you can review what was captured before the interrupt.

**Warning.** A forced kill (*kill -9*) bypasses the cleanup trap. If that happens, the deployed agent markdown files may be left in the target repo's *.claude/agents/* directory. Remove them manually before committing any work in the target repo.

## 5.5 Performance and token metrics

Every run writes *quality-audit.json* with per-agent metrics: duration, tokens consumed, tool calls, findings raised, and any quality auditor flags (batched findings, give-up detected, output under 500 characters, and so on). Use these metrics to identify slow agents, expensive providers, and agents that consistently underperform on your codebase.

The canonical location of these metrics inside the run bundle is:

```
qa-data/<project>/runs/<runId>/quality-audit.json
qa-data/<project>/runs/<runId>/coverage-report.json
qa-data/<project>/runs/<runId>/meta.json
```

## 5.6 Alerting when agents misbehave

Sparfuchs QA has no built-in push alerting: alerting is delegated to whatever log infrastructure you already use. The recommended pattern is to set *RENDERER=json*, tee the event stream to a log aggregator, and alert on three conditions:

- **status = errored.** The run failed to produce a final report.
- **qualityFlags contains giveUp or batched.** An agent returned a shallow result.
- **duration exceeds expected threshold.** Tuneable per agent in models.yaml.

# 6. Troubleshooting

The fastest way to diagnose a failed or stuck run is to classify the failure first, then apply the matching remedy. The decision tree below routes you to the right section in this chapter.

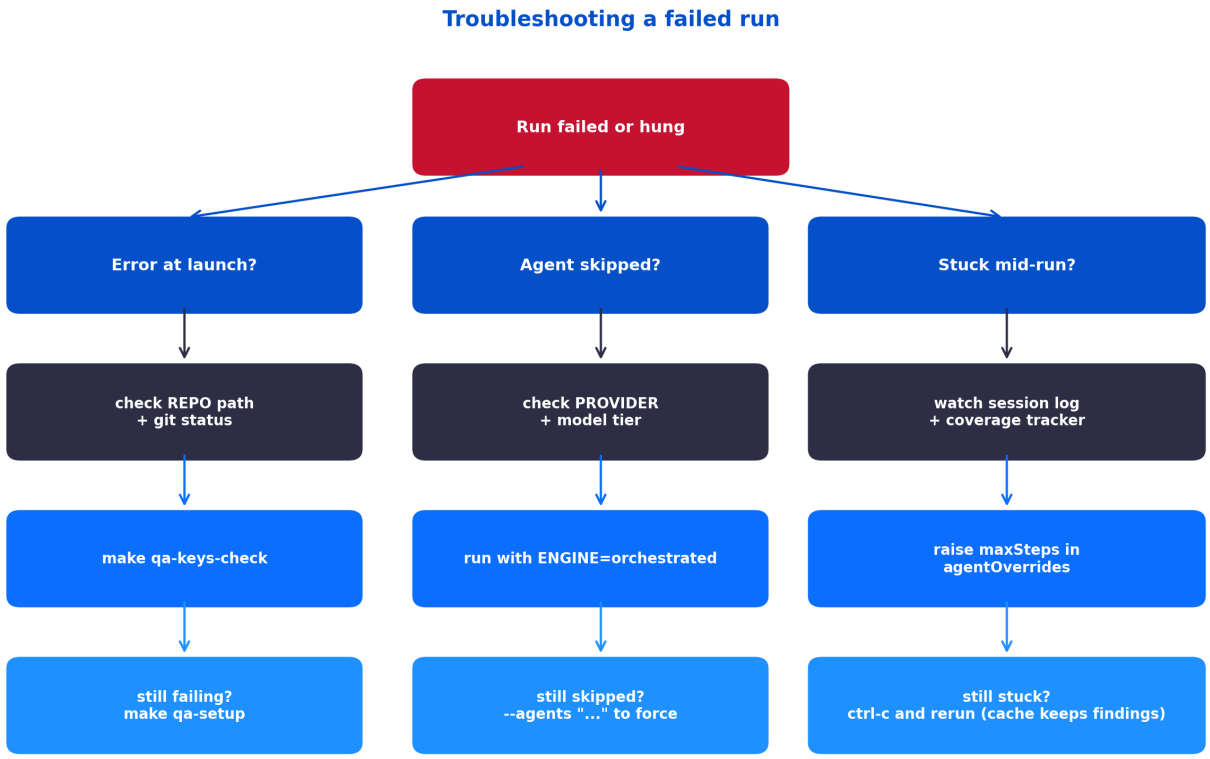


*Figure 6.1 Troubleshooting decision tree.*

## 6.1 Installation and launch errors

| Symptom | Likely cause | Fix |
|---|---|---|
| `make qa-setup` fails | Node below 22 | Install Node 22: `node --version` to verify. |
| `claude: command not found` | Claude Code CLI missing | `npm install -g @anthropic-ai/claude-code` |
| `Error: --repo is required` | Missing REPO flag | Pass an absolute path: `make qa-review REPO=/abs/path` |
| `/path is not a git repository` | Target is not under git | `git init` or pass `ACCEPT_NO_GIT=1` |
| `credential setup failed` | Wizard crashed on interactive input | `npx tsx lib/credentials/setup-wizard.ts` to debug |

| Symptom | Likely cause | Fix |
|---|---|---|
| `Note: overriding target repo's <agent>.md` | Target has a same-named agent | Safe: backed up and restored after the review. |

## 6.2 Canary failures

A failing canary is usually easier to fix than it looks. Each canary returns a *hint* and a *value*. Start with the hint.

| Canary | Typical fix | Approx. time |
|---|---|---|
| hardcoded-credential | Move the literal key to an environment variable or a secrets manager. | 15 to 60 min |
| rbac-bypass | Add a role guard to the offending route, or revise the role hierarchy. | 30 min to 2 hr |
| console-error-leak | Replace *console.error* with a structured logger, or lower the threshold. | 30 min to 2 hr |
| stale-closure | Add the referenced variable to the useCallback dependency array. | 10 to 15 min |
| silent-error-swallow | Add a log line and rethrow, or handle the error at the boundary. | 15 to 45 min |
| coverage-hole | Write a unit test for the file with zero coverage. | 30 min to half a day |
| mock-data-leak | Gate the mock behind a test-only feature flag. | 15 to 45 min |
| todo-density | Resolve or downgrade a few TODOs to notes; raise the threshold only as a last resort. | half a day to 2 days |

## 6.3 Agent skipped or stuck

Some agents are skipped deliberately. For example, *a11y-reviewer* skips when the diff has no frontend files, and the testability preflight can skip agents on files so large they exceed the read window. The skip reason is logged to the live status table.

If you need an agent to run anyway, force it with *AGENTS*:

```
make qa-review REPO=/path/to/project AGENTS="a11y-reviewer"
```

## 6.4 Permission prompts that won't end

Claude Code's *default* permission mode prompts the first time an agent wants to read a file, run a bash command, or write a report. On a large codebase, this can mean hundreds of prompts. Press **a** to allow all reads for the session, or pre-configure broader permissions in your personal *.claude/settings.json*.

## 6.5 Cleanup issues

| Symptom | Remedy |
|---|---|
| Agent files left in target repo | `rm /path/to/project/.claude/agents/*-reviewer.md` |

| Symptom | Remedy |
|---|---|
| Credential file not deleted | `rm /tmp/sparfuchs-qa-creds-*` |
| Empty `.claude/agents/` created in target | `rmdir /path/to/project/.claude/agents /path/to/project/.claude` |

## 6.6 When a finding looks wrong

LLM-generated findings can be incorrect. Before filing a bug:

1. Open the session log and find the agent's raw output for that finding.
2. Verify the file path and line number actually exist.
3. Check the anti-hallucination verdict in *quality-audit.json* for that agent.
4. If the finding is still wrong, mark it as *wont-fix* in your registry with a short justification so delta reports don't re-surface it.

**Tip.** Persistently wrong findings from a specific agent are often a model / prompt mismatch. Try switching that agent to a different provider in *agentOverrides* (see section 4.3).

## 6.7 Error message glossary

| Error | Cause | Action |
|---|---|---|
| AgentIngestionError | Agent emitted malformed JSON | Re-run; if persistent, bug the agent's prompt. |
| Context overflow (finishReason=length) | Token budget exceeded | Raise *maxSteps* or switch to a heavier tier. |
| Integrity mismatch (advisory) | `.claude/agents/.md` changed | Run *make qa-hashes-update* after intentional edits. |
| No providers available | All enabled providers failed auth or detection | Check *make qa-keys-check*; install a CLI fallback. |
| Target is not a git repo | Missing `.git/` or dirty path | `git init` or pass *ACCEPT_NO_GIT=1*. |

# 7. Reviewing and acting on reports

A run produces five human-readable reports. This chapter explains what each one is for, how to read it, and how to translate findings into a concrete day of fixes.

## 7.1 Report bundle at a glance

| Report | Audience | What it contains |
|---|---|---|
| qa-report.md | Everyone | The canonical, severity-ranked list of findings. |
| remediation-plan.md | Engineer fixing bugs | Findings grouped by file, with suggested fixes. |
| observability-gaps.md | Site reliability team | Coverage of the 6 observability dimensions. |
| qa-gaps.md | QA lead | Where the review could not cover, and why. |
| delta-report.md | Release manager | New, recurring, and remediated findings versus the previous run. |
| session-log.md | Deep investigation | Full raw agent transcripts. |

## 7.2 A reading order that works

1. Open *qa-report.md*. Read the executive summary at the top.
2. Scan the Critical section. Resolve or delegate every item.
3. Scan the High section. Triage into *now*, *this sprint*, or *backlog*.
4. Open *delta-report.md*. Look for recurring findings. Recurring items deserve more attention than any new finding.
5. Open *qa-gaps.md*. Note which areas the review could not cover, and whether you need to run a targeted follow-up with *AGENTS=...*.
6. Archive the run's markdown reports alongside your release notes.

## 7.3 The release gate verdict

The *release-gate-synthesizer* aggregates every agent's output into one of three verdicts: **PASS**, **NEEDS CHANGES**, or **BLOCKED**. The verdict is a recommendation, not a hard gate. You retain final authority.

| Verdict | What it means | Typical action |
|---|---|---|
| PASS | No Critical findings; a small number of Medium or Low. | Ship. |
| NEEDS CHANGES | One or more High findings, or several Medium ones in hot files. | Fix the High items, re-run a diff review, then ship. |

| Verdict | What it means | Typical action |
|---|---|---|
| BLOCKED | At least one Critical finding, or an agent error prevented a required check. | Do not ship until the Critical is resolved. |

## 7.4 Generated tests and the graduation ratchet

If the run included generator agents (*crud-tester*, *e2e-tester*, *fixture-generator*, *boundary-fuzzer*), you will find scripts under:

```
sparfuchs-qa/generated/<project>/
crud-tests/users.crud.test.ts
e2e-tests/auth-login-flow.spec.ts
contract-tests/api-users.contract.test.ts
fixtures/user-factory.ts
```

These are **drafts**. Review them, adapt them to your repo's test conventions, and copy the useful ones into your project's test suite. Generated tests are never committed to git automatically.

### 7.4.1 Why the graduation ratchet exists

LLMs hallucinate. Without a defensive system, an AI-generated test suite quickly fills with assertions that pass by accident, test the wrong thing, or pin down hallucinated behavior. The graduation ratchet is the mechanism that prevents that failure mode. Every generated test enters the suite as a **draft**, earns **active** status by surviving human review, and only becomes a **deploy-blocking** (graduated) test after it has passed cleanly on five consecutive builds. Any subsequent flake demotes it back to active.

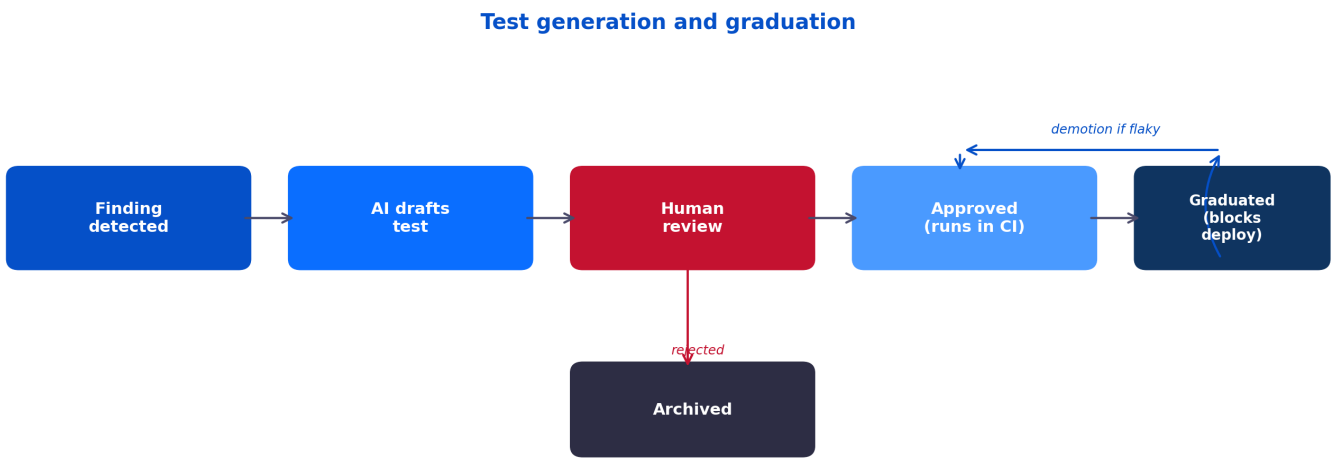


*Figure 7.1 Lifecycle of a platform-generated test.*

### 7.4.2 The five states

| State | How it got here | What it does |
|---|---|---|
| Draft | An agent produced it based on a finding | Sits in *generated/*; does not run in CI. |

| State | How it got here | What it does |
|---|---|---|
| Approved | A human clicked approve in the review dashboard | Eligible to run. Not yet trusted. |
| Active | An operator added it to the project's test suite | Runs in CI. Failures are reported but do not block deploys. |
| Graduated | Passed cleanly on 5 consecutive builds | Treated as a first-class deploy blocker, same as a canary. |
| Demoted (back to Active) | A graduated test started flaking | Automatically demoted. No deploy block. Back on probation. |

### 7.4.3 A concrete example

The *code-reviewer* flags a null-pointer risk in *UserService.ts*. The *fixture-generator* and *crud-tester* produce a draft test that creates a user without an email and expects the service to return a validation error. A reviewer approves the draft, an operator copies the file into *src/services/UserService.test.ts*, and CI runs it on each push. On the fifth consecutive passing build, the promoteTest function graduates the test. Three weeks later, a flaky mock causes it to fail intermittently; the demote circuit breaker kicks in, and the test returns to active. The release gate synthesizer reports the demotion in the next run's report so the team can investigate the flake rather than delete the test.

**Note.** The graduation ratchet is one of seven anti-hallucination layers described in the Admin Guide (chapter 8.8). Combined with schema validation, import verification, and cross-provider auditing, it is why we believe the generated test suite is trustworthy enough to gate deploys.

# 8. Frequently asked questions

### Do I need an API key, or are local CLIs enough?

Local CLIs (claude, gemini, codex) work. The orchestrator auto-detects them from PATH. API keys are recommended for longer runs because they give the orchestrator full tool-use observability. If you only have local CLIs, set *ENGINE=orchestrated PROVIDER=claude-cli*.

### Will the toolkit ever push to my git remote?

No. Sparfuchs QA never commits, never tags, and never pushes. It only reads.

### Does Sparfuchs QA phone home?

No. External report push has been removed. All artifacts are written locally under *qa-data/* and *qa-reports/*. The only network calls are to the LLM providers you explicitly enabled.

### Can I run it offline?

Partially. Canaries are fully offline: *make qa-quick* runs without any network. Agent-based reviews require an LLM provider, which today means either the internet or a locally hosted model via *openclaw*.

### How long does a full audit take?

Typically 30 to 60 minutes on a repo of ~50k lines, with the *balanced* coverage strategy and the default provider chain. Larger repos trigger chunking, which parallelizes work across chunks.

### How do I share a report with my team?

The markdown files in *qa-reports/* are self-contained. Paste them into your issue tracker, attach them to a PR, or publish them to a docs site. The web version of this guide links to a sample report.

### Can two operators run reviews on the same machine concurrently?

Yes. Every run has a unique *runId* that namespaces its artifacts. The only contention is LLM rate limits.

### What if my project is not in TypeScript?

Canaries are TypeScript-first but many agents work on any language. The testability preflight will flag agents that have nothing to analyze in your language (e.g., *stale-closure* is a React check) and skip them.

### How do I stop a specific finding from recurring?

Open the finding in the registry and set its lifecycle to *wont-fix*. Add a short justification. The delta reporter will continue to show the finding but will not flag it as *recurring*.

### Is there a GUI?

Not in this repo. A separate dashboard repo exists internally for visualizing findings. Ask your administrator if your team has access.

### How do I know which agent produced a finding?

Every line in *qa-report.md* is prefixed with the agent name in square brackets, for example *[security-reviewer]*. Open the session log for the raw transcript.

### Can I run the same review twice and get the same result?

Not exactly. Agents use LLM inference at low temperature (0.1), so findings are broadly stable but not bit-for-bit identical between runs. If reproducibility matters, pin the provider and tier in *config/models.yaml*.

### Can I gate my CI pipeline on Sparfuchs QA?

Yes. *make qa-quick* exits non-zero on any failing canary, so it plugs into any CI system. The orchestrated review is heavier; most teams run the diff review as a required PR check and reserve full audits for a nightly or weekly schedule. A **BLOCKED** verdict from the release gate is the right signal to fail the job.

### How do I convince my security team this is safe to run on our proprietary code?

Three things usually close the conversation. First, point them at the Admin Guide's Agent Sandboxing and Containment section (chapter 6.6), which covers filesystem jail, Bash allow-list, classification routing, and hook-based pre-write scanning. Second, show them the Known Gaps table (Admin chapter 8.5) so they see the honest list and the interim mitigations. Third, set *dataClassification: restricted* during their evaluation so source never leaves the host. Most security teams greenlight after those three artifacts.

### Can I use this on a monorepo with 2M+ LOC?

Yes, but not the way you would run it on a 50k-line repo. Use module scoping (*MODULE="apps/<name>"*) to focus one app at a time. Downgrade the coverage strategy to *balanced* or *sweep*. Switch from the *full* preset to *standard*. The testability scanner will chunk the work into groups of roughly 25 files. Expect hours, not minutes, on the first full audit. See Admin Guide chapter 9.5 on scaling.

# 9. Glossary

**Adapter.** A small integration layer in *lib/orchestrator/adapters/* that lets the orchestrator drive a specific provider or CLI (api, claude-cli, gemini-cli, codex-cli, openclaw).

**Agent.** A markdown file in *.claude/agents/* that defines a specialist: prompt, allowed tools, model tier, and expected output shape.

**Agent-data envelope.** A JSON document an agent emits to hand off data to a later agent. Lives at *agent-data/<agent>.output.json*.

**Canary.** A fast, deterministic assertion that runs in under 500 milliseconds. Auto-discovered from *canaries/*.canary.ts*.

**Chunker.** Splits a large target repo into directory-based groups of roughly 25 files so each agent can stay within context limits.

**Coverage strategy.** A *config/models.yaml* setting that trades depth for cost: *sweep*, *balanced*, *thorough*, *exhaustive*.

**Data classification.** A *config/models.yaml* setting that gates which providers may receive repo data: *public*, *internal*, *restricted*.

**Delta report.** A run-over-run comparison of findings: new, recurring, remediated.

**Engine.** Direct mode (single Claude CLI) or Orchestrated mode (multi-provider, *ENGINE=orchestrated*).

**Finding.** A single reported issue, with agent, severity, file, rule, and fix suggestion.

**Finding registry.** The local JSONL store of findings with lifecycle state, in *qa-findings-manager*.

**Give-up detector.** Quality auditor check that flags agents with zero tool calls who claim inability.

**Graduation.** The promotion of a generated test from Active to Graduated after 5 consecutive passes; graduated tests block deploys on failure.

**Hallucination risk.** A per-agent quality auditor score on a 0 to 100 scale (0 = clean, 100 = highly suspect). Derived from path verification, tool usage patterns, output size, and cross-provider audit. Scores above 40 produce a warning; scores above 70 trigger an automatic re-run with a different provider.

**Model tier.** *heavy* (complex reasoning), *mid* (general), *light* (fast classification). Maps agent frontmatter to provider model.

**Orchestrator.** The engine in *lib/orchestrator/* that parses agents, runs them, audits outputs, and writes artifacts.

**ReAct loop.** Reason-and-Act: an agent alternates reasoning (plan) and acting (tool call) until it produces a final result.

**Release gate.** The synthesizer agent that aggregates all other agents into *PASS*, *NEEDS CHANGES*, or *BLOCKED*.

**Run bundle.** The per-run directory under *qa-data/<project>/runs/<runId>/*.

**Testability preflight.** A scanner that decides which agents are viable on this codebase and which files are too large or malformed to audit.

**Tool calling.** An agent's ability to invoke local tools (Read, Grep, Bash, Write) during its run, controlled per-agent in *models.yaml*.

**Verdict.** The final release-gate decision on a run: *PASS*, *NEEDS CHANGES*, *BLOCKED*.

# 10. Appendices

## Appendix A. Command reference

| Command | Purpose |
|---|---|
| `make qa-setup` | Install npm dependencies. |
| `make qa-quick` | Run the 15 canary suite (~10s). |
| `make qa-watch` | Re-run canaries on file changes. |
| `make qa-review REPO=...` | Diff review of changed files. |
| `make qa-review REPO=... FULL=1` | Full audit of every source file. |
| `make qa-review ... AUTH=1` | Full audit with credential setup. |
| `make qa-review ... AGENTS="a,b"` | Selective agent run. |
| `make qa-review ... MODULE="apps/x"` | Scope to a subdirectory. |
| `make qa-training REPO=...` | Generate training materials. |
| `make qa-docs REPO=...` | Generate architecture docs. |
| `make qa-stubs REPO=...` | Hunt for stubs and dead integrations. |
| `make qa-build-check REPO=...` | Build verification only. |
| `make qa-schema-check REPO=...` | DB schema validation. |
| `make qa-delta PROJECT=...` | Compare the last two runs. |
| `make qa-evolve-v2 PROJECT=...` | Analyze patterns and suggest thresholds. |
| `make qa-cleanup PROJECT=...` | Archive old runs beyond the last 10. |
| `make qa-cache-status PROJECT=...` | Show the file audit cache. |
| `make qa-cache-reset PROJECT=...` | Clear the file audit cache. |
| `make qa-keys-check` | List API keys in the OS keychain. |
| `make qa-keys-setup` | Instructions for storing keys. |
| `make qa-creds-list` | List saved test credential profiles. |
| `make qa-creds-store NAME=staging-admin` | Store a new test credential profile. |
| `make qa-creds-show NAME=staging-admin` | Show a test credential profile. |
| `make qa-creds-delete NAME=staging-admin` | Delete a test credential profile. |
| `make qa-hashes-update` | Regenerate *agent-hashes.json*. |
| `make qa-sca` | Supply chain check + SBOM. |
| `make qa-verify` | SCA dry run (no writes). |

## Appendix B. Useful environment variables

| Variable | Purpose |
|---|---|
| `REPO` | Target repo path (required for review modes). |
| `PROJECT` | Project slug used in artifact paths. |
| `PERSON` | Reviewer name recorded in report metadata. |
| `URL` | Target app URL, recorded in metadata. |
| `FULL=1` | Switch diff review into full audit. |
| `AUTH=1` | Run credential setup wizard. |
| `AGENTS="a,b,c"` | Run only the named agents. |
| `MODULE="apps/web"` | Scope to a subdirectory. |
| `JOURNEY="onboarding"` | Training journey scope. |
| `ENGINE=orchestrated` | Use multi-provider engine. |
| `PROVIDER=xai\|google\|anthropic\|openai\|claude-cli\|gemini-cli\|codex-cli\|openclaw` | Pick a provider in orchestrated mode. |
| `RENDERER=tty\|log\|json` | Status renderer. |
| `ACCEPT_NO_GIT=1` | Allow non-git targets. |
| `TARGET_REPO=/abs/path` | Override target path for canaries. |
| `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `XAI_API_KEY`, `GOOGLE_GENERATIVE_AI_API_KEY`, `GEMINI_API_KEY` | API credentials, read if keychain is unavailable. |

# Appendix C. Printable pre-push checklist

1. Run *make qa-quick*. Confirm 15/15 pass.
2. Run *make qa-review REPO=.* for the current repo.
3. Open *qa-reports/<date>_<project>_qa-report.md*.
4. Resolve every Critical. Triage High into now/sprint/backlog.
5. Open *delta-report.md*. Investigate any recurring item.
6. Commit and push.

# Appendix D. Support and feedback

File an issue in the sparfuchs-qa repository for bug reports or documentation requests. Use the *docs* label for anything about this guide. For security issues, email the maintainer listed in *MAINTAINERS.md*.

# Appendix E. Changelog

Version 1.0, April 22, 2026

- First published edition.
- Covers canary suite, orchestrated review, training and docs modes.

- Based on sparfuchs-qa repo state as of 2026-04-13 (session-cleanup agent update).

Planned for v1.1

- Worked example chapter (a full run on a sample e-commerce repo).
- Screenshots of the TTY renderer.
- Expanded troubleshooting for the openclaw adapter.

**Note.** This roadmap is advisory and not a commitment. Features and dates may change without notice. For the current feature set, refer to the most recent release notes.

# Appendix F. Documentation maintenance plan

The User Guide is owned by the documentation team. The update cadence is quarterly for normal changes and immediate for any removed or renamed command. Every Sparfuchs QA release includes a documentation diff as part of the release gate verdict. If *doc-reviewer* flags drift between source and this guide, the release is marked *NEEDS CHANGES* until the guide is updated.